

American University of Sharjah
College of Engineering
COE 420L – Software Engineering Lab

Lab 5: Object Modeling and UML Class Diagrams

Project: Lost and Found Property Claim System

Team Members:
Hamda Albahri — g00100284
Hamda Bahakeem — g00100146
Mahra Alhammadi — g00098713

GitHub Repository:
https://github.com/COE420-Group1-F26/LostandFound-System

Date of Submission: October 27, 2026

Team Contributions:

Each member worked on their own component in Part A and on a separate part of the integration in Part B. Every member's work can be seen in the commit history of the GitHub repository (each commit message includes the member's name).

Task	Mahra Alhammadi (g00098713) — Member 1	Hamda Albahri (g00100284) — Member 2	Hamda Bahakeem (g00100146) — Member 3
Component modeled (Part A)	Users, lost item reports and search (P1, P2, P4)	Found item reports, matching and notifications (P3, P5)	Claims (P6)
Individual classes	C01 User, C02 ItemOwner, C03 Finder, C04 CampusSecurityOfficer, C05 LostItemReport, C06 ItemSearch	C10 FoundItemReport, C11 ItemPhoto, C12 ReportStatus, C13 MatchSuggestion, C14 MatchingService, C15 Notification	C20 Claim, C21 ClaimStatus, C22 VerificationRecord, C23 DisputeCase, C24 ReleaseRecord
Lab 5 folder structure	Yes	—	—
Exercise 1 – Individual class inventory	Yes	Yes	Yes
Exercise 2 – Individual UML class diagram	Yes	Yes	Yes
Exercise 3 – Written review of the other two models	Yes	Yes	Yes
Exercise 3 – Class inventory (consolidation record) and integration notes	Yes	—	—
Exercise 3 – Consolidated UML class diagram	—	Yes	—
Class model consistency and traceability check	—	—	Yes
Changes made to earlier labs	—	—	Yes
Final report and submission	Yes	Yes	Yes

Exercise 1 – Individual Class Inventory:

Each team member modeled one component of the Lost and Found system, based on the use cases, functional requirements and data stores from Labs 3 and 4. Member 1 modeled users, lost item reports and search (P1, P2, P4), Member 2 modeled found item reports, matching and notifications (P3, P5), and Member 3 modeled claims (P6). Each member used their own range of class IDs (C01–C09, C10–C19 and C20–C29) so that IDs did not clash, and classes owned by another member were only referenced by name.

Member 1 – Mahra Alhammadi (g00098713): Users, Lost Item Reports and Search

Component modeled: Users and authentication, lost item reports, and searching found items

Level-0 processes: P1 Authenticate Users, P2 Manage Lost Item Reports, P4 Search Found Items

Data stores: D1 User Accounts, D2 Lost Item Reports (D3 Found Item Reports is read by the search)

Classes:

Class ID	Class Name	Purpose/Responsibility	Related Use Case(s)	Contributor
C01	User (abstract, entity)	Holds the account data kept in D1 (AUS ID, name, email, password hash) and provides login, logout and password change for every person who uses the system.	UC-01	Mahra Alhammadi
C02	ItemOwner (entity)	The registered account of a person who lost an item. Submits a lost item report and searches the found items.	UC-01, UC-02, UC-03	Mahra Alhammadi
C03	Finder (entity)	The registered account of a person who found an item and hands it in through the system.	UC-01	Mahra Alhammadi
C04	CampusSecurityOfficer (entity)	The registered account of security staff who log in to manage found items and decide claims.	UC-01	Mahra Alhammadi
C05	LostItemReport (entity)	The record of one lost item: name, category, description, where and when it was lost, private identifying details and the owner's contact email. Stored in D2.	UC-02	Mahra Alhammadi
C06	ItemSearch (control)	The criteria an owner enters (keyword, category, date range, location) and the operation that looks through the found items, so it carries out the search task.	UC-03	Mahra Alhammadi

Why each class is a class (not just an attribute):

Class	Justification
User	The three actors share the same account data and the same login behaviour (UC-01), so it is written once in a parent class.
ItemOwner	Has its own behaviour (reporting lost items and searching, UC-02 and UC-03) that the other roles do not have.
Finder	A separate actor in the use case model with its own role (submits found reports); it needs its own class so the found-item side can associate with it.
CampusSecurityOfficer	A separate actor that decides claims and manages found items; those responsibilities are modeled with Member 2 and Member 3.
LostItemReport	A lost report has its own data and life cycle (submit, update, withdraw) and is stored in D2, so it is a class and not an attribute of ItemOwner.
ItemSearch	A search has several criteria and one operation that uses them together; modeling it as a class keeps the owner's class small.

Object types:

- **Entity objects** (persistent information tracked by the system): User, ItemOwner, Finder, CampusSecurityOfficer, LostItemReport.

- **Control objects** (tasks performed by the system): ItemSearch.
- **Boundary objects** (interaction between user and system): none are modeled. The login, lost report and search screens belong to the user interface design, and the team decides in Exercise 3 whether any boundary class is needed.
- **Actors vs classes:** the actors (Item Owner, Finder, Campus Security Officer) are outside the system. ItemOwner, Finder and CampusSecurityOfficer here are the registered accounts the system stores in D1 User Accounts, so they are inside the system.

Inheritance used:

ItemOwner, Finder and CampusSecurityOfficer inherit from the abstract class User. This is a real is-a relationship: each of them is a user that logs in with the same email and password (UC-01), and an ItemOwner, Finder or CampusSecurityOfficer can be used wherever a User is expected (substitutability).

Relationships:

- LostItemReport is a **composition** of ItemPhoto (filled diamond): a photo of a lost item lives and dies with its report.
- ItemOwner is associated with LostItemReport (reports, 1 to 0..*) and with ItemSearch (runs, 1 to 0..*).
- ItemSearch is associated with Member 2's FoundItemReport (looks through, 0..* to 0..*).
- LostItemReport has a **dependency** on the enumeration ReportStatus, because it uses it as the type of its status attribute.

Classes of other members used for reference only:

Class	Owner
FoundItemReport	Member 2
ItemPhoto	Member 2
ReportStatus (enumeration)	Member 2

Open design questions (to settle in Exercise 3):

1. Does a lost item report need a status? It is shown as status: ReportStatus, reusing Member 2's enumeration. If lost reports use different states, the team will define a separate enumeration.
2. A lost item report has one photo (0..1), while Member 2's found reports may allow several, and ItemPhoto cannot be a composition of two different classes. The team may give each report its own photo class, or create a shared parent class for LostItemReport and FoundItemReport, which both hold category, description, location, date and photo.
3. The approve and reject claim operations on CampusSecurityOfficer may move to Member 3's claim classes during integration.

Member 2 – Hamda Albahri (g00100284): Found Item Reports, Matching and Notifications

Component modeled: Found Item Reports, Matching and Notifications

Level-0 Processes: P3 Manage Found Item Reports, P5 Match Items and Notify Owners

Data stores: D3 Found Item Reports, D4 Match Suggestions

Requirements covered: FR-04, FR-06 to FR-10

Classes:

Class ID	Class Name	Purpose/Responsibility	Related Use Case(s)	Contributor
C10	FoundItemReport	Stores information about a found item, including its category, description, color, location and date/time found, and its current status. Stored in D3.	UC-06, UC-07, UC-08, UC-10	Hamda Albahri
C11	ItemPhoto	Represents a photo attached to a found item report to help identify the item (up to 3 per report, FR-07).	UC-06, UC-07	Hamda Albahri

Class ID	Class Name	Purpose/Responsibility	Related Use Case(s)	Contributor
C12	ReportStatus	Defines the allowed report statuses: Pending, Matched, Claimed, Returned, and Withdrawn.	UC-08, UC-10	Hamda Albahri
C13	MatchSuggestion	Represents a possible match between a lost item report and a found item report, including a matching score and rank. Stored in D4.	UC-09, UC-04	Hamda Albahri
C14	MatchingService	Compares lost and found item reports and generates ranked match suggestions.	UC-09	Hamda Albahri
C15	Notification	Represents the email notification sent to an item owner when a possible match is identified.	UC-04	Hamda Albahri

Why Each Class Is Needed:

Class	Justification
FoundItemReport	A found item requires its own record containing multiple details and a status (FR-06, FR-10), and it has its own life cycle: submit, edit, withdraw (FR-08).
ItemPhoto	A report may contain up to 3 photos that help users recognize found items (FR-07). Each photo has its own file name and upload date, so it is more than one attribute.
ReportStatus	An enumeration ensures that reports use only valid status values (FR-10, D3).
MatchSuggestion	Each suggested match needs to store its associated reports, matching score and rank (FR-09, D4).
MatchingService	Matching requires separate logic to compare reports and generate ranked suggestions (FR-09).
Notification	Notifications require their own message details, link and delivery information (FR-04).

Object Types:

- **Entity objects:** FoundItemReport, ItemPhoto, MatchSuggestion, Notification.
- **Control objects:** MatchingService.
- **Enumeration:** ReportStatus.

Inheritance:

No inheritance is used in this component. ReportStatus is an enumeration (a fixed list of values), not a parent class. FoundItemReport and LostItemReport share some attributes, but the requirements do not yet justify a shared parent class, so this is left for the team to decide in Exercise 3.

Relationships:

- FoundItemReport is a **composition** of ItemPhoto (has, 1 to 0..3): a photo cannot exist without its report.
- Finder submits FoundItemReport (1 to 0..*).
- MatchingService creates MatchSuggestion (1 to 0..*).
- MatchSuggestion points to one FoundItemReport and is for one LostItemReport (0..* to 1 each).
- Notification announces one MatchSuggestion (0..1 to 1) and is sent to one ItemOwner (0..* to 1).
- FoundItemReport has a **dependency** on ReportStatus (uses it as the type of its status attribute).

Classes of Other Members Used for Reference Only:

Class	Owner
Finder	Member 1
ItemOwner	Member 1
LostItemReport	Member 1

Open Design Questions (for Exercise 3):

1. Member 1 also links ItemPhoto to LostItemReport by composition. One class cannot be a composition part of two different wholes, so the team must choose: separate photo classes, or a shared parent class for the two reports.
2. CampusSecurityOfficer has no direct association with FoundItemReport here, because UC-08 and UC-10 belong to the Finder. The officer changes the status through the claim process (P6, Member 3).
3. UC-09 names the Finder as the primary actor, but FR-04 sends the email to the owner of the lost report. The Notification follows FR-04; the Finder sees matches through the found report.

Member 3 – Hamda Bahakeem (g00100146): Claims

Component modeled: Claims — submitting a claim, checking it, approving or rejecting it, handling disputes, and releasing the item

Level-0 processes: P6 Process and Decide Claims

Data stores: D5 Claims (my part also reads D3 Found Item Reports to see the item details)

Requirements covered: FR-05, FR-11 to FR-16

Classes:

Class ID	Class Name	Purpose/Responsibility	Related Use Case(s)	Contributor
C20	Claim	When an owner says "this found item is mine". It keeps their AUS ID, the private details they give to prove it, and the claim status. Saved in D5.	UC-05, UC-11, UC-12, UC-13	Hamda Bahakeem
C21	ClaimStatus (enumeration)	The only statuses a claim can have: PENDING_VERIFICATION, APPROVED, REJECTED, RELEASED.	UC-05, UC-12, UC-13	Hamda Bahakeem
C22	VerificationRecord	One decision by a security officer on a claim (approved or rejected), with who decided, notes, and the date and time. All of these together are the claim history.	UC-12, UC-13, UC-15	Hamda Bahakeem
C23	DisputeCase	Used when two or more people claim the same item. The officer compares them, approves one and rejects the others.	UC-14	Hamda Bahakeem
C24	ReleaseRecord	Made when an approved claim is ready for pickup, and finished when the owner actually gets the item.	UC-12	Hamda Bahakeem

Why each class is a class (not just an attribute):

Class	Justification
Claim	A claim has its own info (AUS ID, details, date, status) and goes through its own steps from submitted to decided. One item can have more than one claim, so it can't just be a field inside FoundItemReport.
ClaimStatus	FR-05, FR-12 and FR-13 say exactly which statuses are allowed ("Pending Verification", "Approved", "Rejected"), so an enumeration makes sure nothing else is used. It's different from Member 2's ReportStatus because that one is about the item, this one is about the claim.
VerificationRecord	FR-15 needs a history of every approve/reject, who did it and when. A claim can get more than one decision, so one field on Claim isn't enough.
DisputeCase	UC-14 and FR-16 need all the claims for the same item grouped together and decided once. The dispute has its own dates and result that don't belong to just one claim.
ReleaseRecord	FR-14 says an item can only be released after the claim is approved. The release has its own info (when it was ready, when it was picked up, who picked it up) and only approved claims have it, so I kept it separate from Claim.

Object types:

- **Entity objects:** Claim, VerificationRecord, DisputeCase, ReleaseRecord (all saved in D5 Claims).

- **Enumeration:** ClaimStatus.
- **Control objects:** none. The officer makes decisions through the methods in Claim and DisputeCase, so an extra control class wouldn't add anything.
- **Boundary objects:** none. The claim form and officer screens are part of the UI design.

Inheritance:

I didn't use inheritance. ClaimStatus is an enumeration (just a list of values), not a parent class. My other classes are different kinds of records and none of them "is a" type of another, so a parent class wouldn't make sense.

Relationships:

- ItemOwner submits Claim (1 to 0..*): one owner can make many claims, but each claim is from one owner (FR-05).
- Claim is for FoundItemReport (0..* to 1): many people can claim the same item, but each claim is for one item (FR-05, FR-11).
- Claim has VerificationRecord, **composition** (1 to 0..*): the decisions belong to that claim and make up its history (FR-15).
- CampusSecurityOfficer records VerificationRecord (1 to 0..*): every decision shows which officer made it (FR-15).
- DisputeCase groups Claim (0..1 to 2..*): a dispute needs at least 2 claims, and a claim can only be in one dispute (UC-14, FR-16).
- CampusSecurityOfficer resolves DisputeCase (0..1 to 0..*): an open dispute has no officer yet, a solved one has one (FR-16).
- Claim ends with ReleaseRecord (1 to 0..1): only approved claims get a release record (FR-14).
- Claim **uses** ClaimStatus (dependency): it's the type of the status attribute.

Classes of other members used for reference only:

Class	Owner
ItemOwner	Member 1
CampusSecurityOfficer	Member 1
FoundItemReport	Member 2

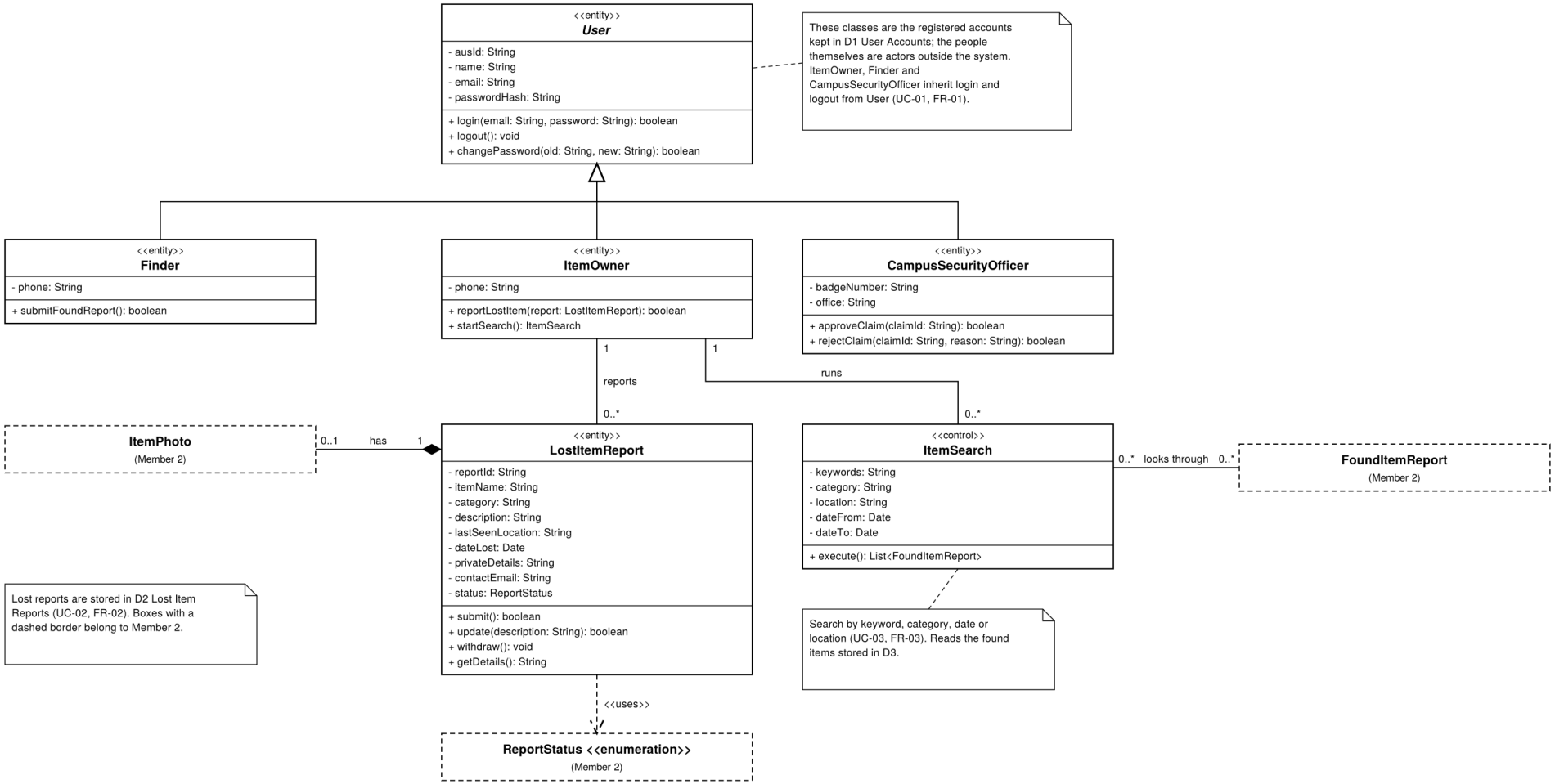
Open design questions (to settle in Exercise 3):

1. Member 1 has approveClaim() and rejectClaim() in CampusSecurityOfficer, and I have approve() and reject() in Claim. We should keep only one, or have the officer's methods just call the Claim ones.
2. ReleaseRecord doesn't have its own use case, it's there for UC-12 and FR-14. We need to decide if it stays a class or becomes a few attributes in Claim.
3. When a claim is approved or the item is picked up, the item's status (Member 2's ReportStatus) should change to CLAIMED and then RETURNED. We need to agree which class does this.
4. The task division suggested a DISPUTED status, but no requirement mentions it, so I left it out. A disputed claim just stays PENDING_VERIFICATION until the dispute is solved.

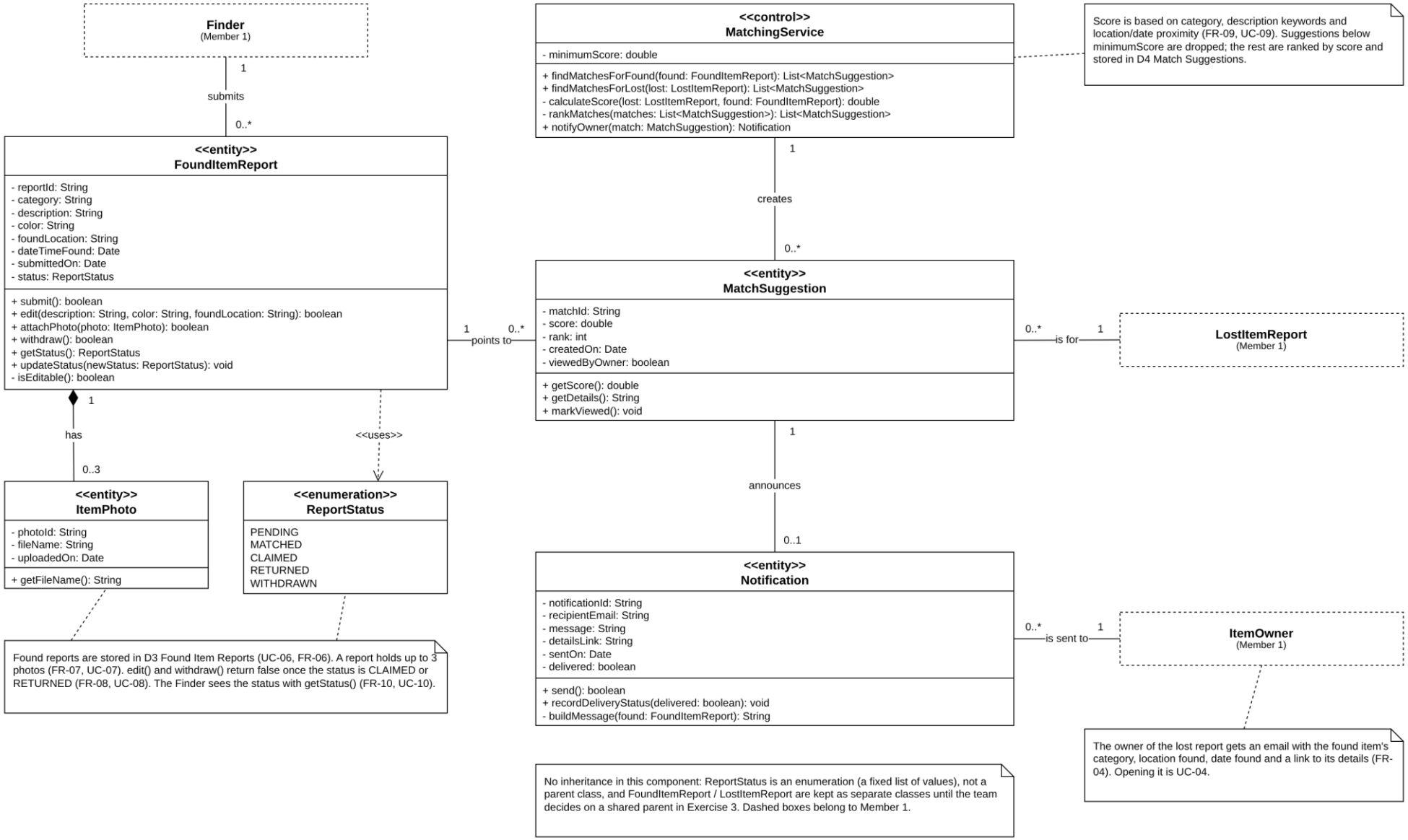
Exercise 2 – Individual UML Class Diagrams:

Each member drew a UML class diagram for their own component in draw.io. Every class shows its attributes and methods with visibility, data types, parameters and return types, and every association has a name and a multiplicity at both ends. Classes owned by another member are drawn as dashed boxes with only the class name and the owner, so no class is duplicated.

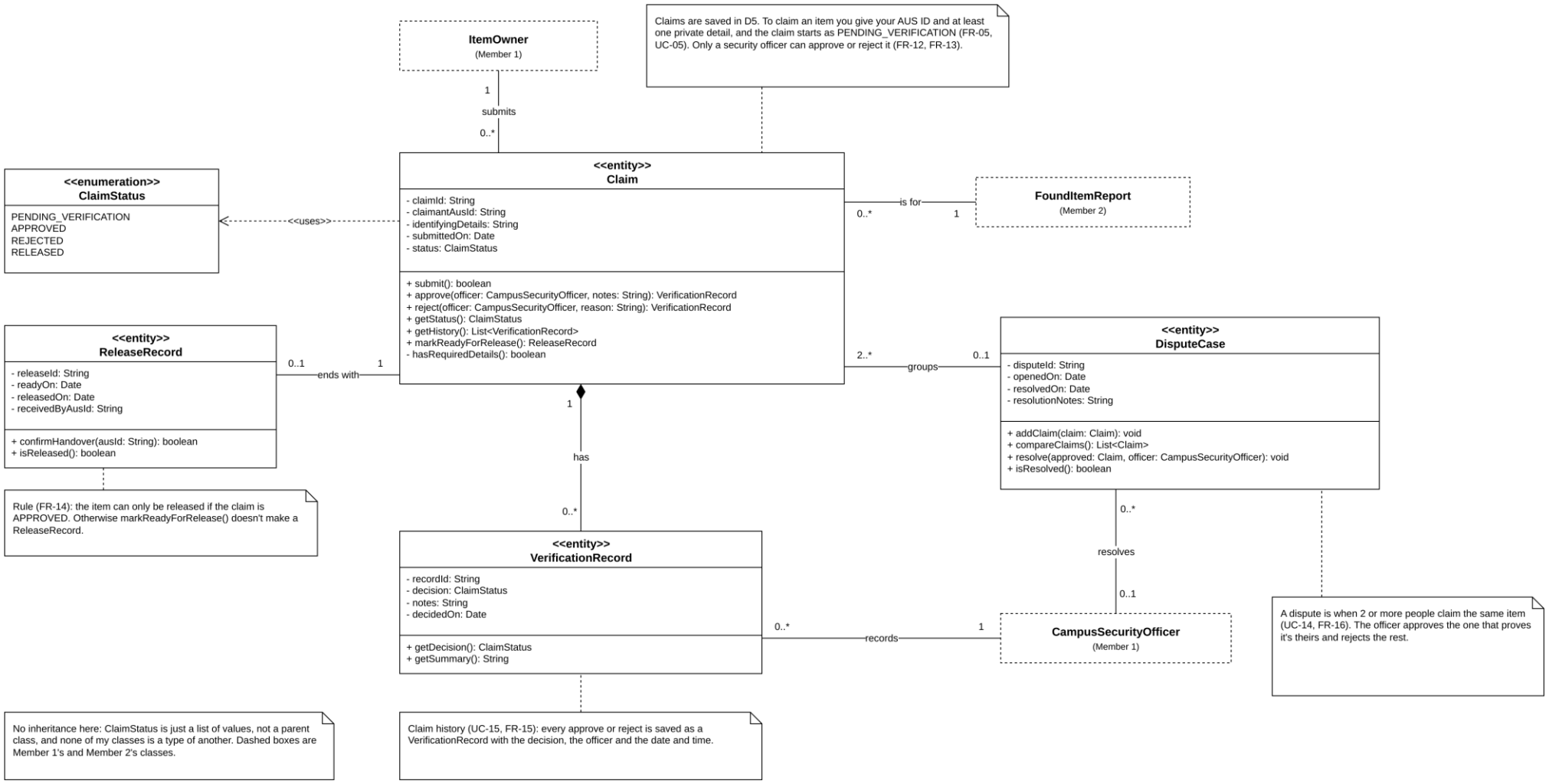
Member 1 – Mahra Alhammadi (g00098713) – Individual UML Class Diagram:



Member 2 – Hamda Albahri (g00100284) – Individual UML Class Diagram:



Member 3 – Hamda Bahakeem (g00100146) – Individual UML Class Diagram:



Exercise 3 – Model Integration and Consolidated Class Diagram:

After the oral, each member reviewed the other two individual models in writing (Review_of_Others.md in each member's folder on GitHub). Member 1 then combined the three models into one class inventory and recorded every integration decision, Member 2 drew the consolidated class diagram, and Member 3 checked the final model against the requirements and earlier labs.

Class Inventory and Consolidation Record:

Inputs: the three individual inventories and class diagrams (Project/Object_Modeling/Individual_Models/Member1 to Member3), the three written reviews (Review_of_Others.md by each member), and the open design questions in each inventory. Every decision is explained in Integration_Notes.md.

Class IDs are kept from the individual inventories. C07 is new (added during integration). C03 was merged into C02.

1. Approved classes:

Class ID	Class Name	Object Type	Purpose/Responsibility	Related Use Case(s)	Source (Member)
C01	User (abstract)	Entity	Account data kept in D1 User Accounts (AUS ID, name, email, password hash); create account, log in, log out.	UC-01	Member 1
C02	CampusMember	Entity	The account of an AUS student or staff member. Plays the owner role (reports lost items, searches, claims) and the finder role (reports found items). Replaces ItemOwner and Finder.	UC-01 to UC-10	Member 1 (merged during integration)
C04	CampusSecurityOfficer	Entity	The account of a security officer. Views pending claims and the claim history, records decisions and resolves disputes.	UC-01, UC-11 to UC-15	Member 1
C05	LostItemReport	Entity	One lost item report, with private identifying details not shown to other users and the owner's contact email. Stored in D2.	UC-02	Member 1
C06	ItemSearch	Control	Holds the search filters (keyword, category, location, date range) and runs the search over the found items.	UC-03	Member 1
C07	ItemReport (abstract)	Entity	Data and operations shared by lost and found reports: category, description, location, date and photos.	UC-02, UC-06, UC-07	Added during integration
C10	FoundItemReport	Entity	One found item report, with its color and status; can be edited or withdrawn until it is claimed. Stored in D3.	UC-06, UC-07, UC-08, UC-10	Member 2
C11	ItemPhoto	Entity	A photo of an item. Exists only as part of its report.	UC-02, UC-07	Member 2
C12	ReportStatus	Enumeration	Status of a found item: PENDING, MATCHED, CLAIMED, RETURNED, WITHDRAWN.	UC-08, UC-10	Member 2
C13	MatchSuggestion	Entity	A possible pair of one lost and one found report, with a score and rank. Stored in D4.	UC-04, UC-09	Member 2

Class ID	Class Name	Object Type	Purpose/Responsibility	Related Use Case(s)	Source (Member)
C14	MatchingService	Control	Compares lost and found reports, ranks the suggestions and triggers the owner's notification.	UC-09	Member 2
C15	Notification	Entity	The email sent to the owner of a lost report when a possible match is found.	UC-04	Member 2
C20	Claim	Entity	A request to collect a found item, with the AUS ID and identifying details entered by the claimant. Stored in D5.	UC-05, UC-11, UC-12, UC-13	Member 3
C21	ClaimStatus	Enumeration	PENDING_VERIFICATION, APPROVED, REJECTED.	UC-05, UC-12, UC-13	Member 3
C22	VerificationRecord	Entity	One approve or reject decision, with the officer, notes and date and time. Together they form the claim history.	UC-12, UC-13, UC-15	Member 3
C23	DisputeCase	Entity	Groups two or more claims for the same item so the officer can compare them and approve one.	UC-14	Member 3
C24	ReleaseRecord	Entity	Created when an approved claim is ready for pickup; completed when the item is handed over.	UC-12	Member 3

Total: 17 classes (13 entity, 2 control, 2 enumeration).

2. Duplicate or overlapping classes identified and resolved:

Classes	Members	Resolution	Reason
ItemPhoto as a composition part of LostItemReport and of FoundItemReport	Member 1, Member 2	ItemPhoto becomes a composition part of the new parent class ItemReport (1 to 0..3), with a note limiting lost reports to 1 photo.	A class cannot be the part of two different compositions. Raised by all three members.
LostItemReport and FoundItemReport	Member 1, Member 2	Shared attributes and operations moved to the abstract class ItemReport; each subclass keeps only what is different.	Same data (category, description, location, date, photos), and MatchingService compares them through these fields (FR-09). Raised by all three members.
ItemOwner and Finder	Member 1	Merged into CampusMember; "owner" and "finder" are now role names on the associations.	FR-02, FR-03 and FR-05 say "a logged-in user": one AUS account can both lose and find items, which two subclasses cannot represent. Raised by Member 3 and Member 2.
Approve and reject operations in CampusSecurityOfficer and in Claim	Member 1, Member 3	Kept only in Claim (approve and reject take the officer as a parameter).	The operations change the claim status and create the VerificationRecord, which belong to Claim. Raised by all three members.
Status of a report	Member 1, Member 2	Status is kept only on FoundItemReport. LostItemReport has no status.	FR-10 gives statuses to found items only; FR-02 gives lost reports no status. Raised by Member 2 and Member 3.
ClaimStatus.RELEASED, ReportStatus.RETURNED and ReleaseRecord.isReleased()	Member 2, Member 3	RELEASED removed from ClaimStatus. The claim stays APPROVED, the ReleaseRecord records the handover and the found item becomes RETURNED.	Three places recorded the same event, and no requirement mentions a "Released" claim status. Raised by Member 2.

3. Classes removed or merged:

Class	Action	Reason
C02 ItemOwner	Renamed to CampusMember	See section 2. The ID C02 is kept.
C03 Finder	Merged into C02 CampusMember	See section 2. The finder role appears as a role name.

No other class was removed. ReleaseRecord was reviewed as a possible attribute group of Claim and kept (see Integration_Notes.md, section 6).

4. Classes added during integration:

Class	Purpose	Why it was missing
C07 ItemReport (abstract)	Parent class of LostItemReport and FoundItemReport; holds the shared data and owns the photos.	No individual model could add it alone, because the two report classes belonged to different members.

Integration Notes:

This file records the significant changes made while consolidating the three individual class models into one team model. It is based on the three written reviews (Review_of_Others.md by each member). Member 2 draws the consolidated class diagram from section 7; Member 3 checks it in Class_Model_Consistency_Check.md.

Exercise 3 steps covered: review of all individual models (the three Review_of_Others.md files); duplicate classes (Class_Inventory.md section 2); names and responsibilities (section 2 below); missing classes (Class_Inventory.md section 4); attributes and methods (section 3); inheritance (section 4); associations and multiplicities (section 5); redundant classes (Class_Inventory.md section 3); conflicting decisions (section 6); complete system (section 7 covers UC-01 to UC-15).

1. Significant modifications made during integration:

No.	Change	Classes Affected	Reason	Raised by
1	Added the abstract class ItemReport as the parent of LostItemReport and FoundItemReport.	ItemReport, LostItemReport, FoundItemReport	Both reports share the same data, and MatchingService compares them through it (FR-09). This also fixes the photo composition problem.	Member 1, Member 2, Member 3
2	ItemPhoto is now a composition part of ItemReport (1 to 0..3), with a note: lost report at most 1 photo (FR-02), found report up to 3 (FR-07).	ItemPhoto, ItemReport	One class cannot be the part of two compositions.	Member 1, Member 2, Member 3
3	Merged ItemOwner and Finder into CampusMember; "owner" and "finder" become role names on the associations.	CampusMember, and every association that used ItemOwner or Finder	One AUS account can lose and find items (FR-02, FR-03, FR-05, FR-06).	Member 3, Member 2
4	Approve and reject are kept only in Claim; removed from CampusSecurityOfficer.	Claim, CampusSecurityOfficer	The same behaviour was in two classes.	Member 1, Member 2, Member 3
5	Added viewPendingClaims(): List<Claim> and viewClaimHistory(): List<VerificationRecord> to CampusSecurityOfficer.	CampusSecurityOfficer	UC-11/FR-11 and UC-15/FR-15 work across all claims, but every operation in the claim model worked on one claim.	Member 1, Member 2
6	Added createAccount(email: String, password: String): boolean to User and removed changePassword().	User	FR-01 asks for creating an account; no requirement asks for changing the password.	Member 1, Member 3

No.	Change	Classes Affected	Reason	Raised by
7	The ItemOwner–ItemSearch association became a dependency (CampusMember uses ItemSearch).	CampusMember, ItemSearch	A search is not stored in any data store, so there is no lasting link.	Member 3, Member 2
8	Claim.approve() sets the found item's status to CLAIMED; ReleaseRecord.confirmHandover() sets it to RETURNED. Shown as a UML note.	Claim, ReleaseRecord, FoundItemReport	Matches the DFD flow "Item Status Update" from P6 to D3 and the statuses in FR-10. It also makes FR-08 work, because edit() and withdraw() refuse once the item is CLAIMED.	Member 3, Member 2
9	Removed LostItemReport.itemName.	LostItemReport	FR-02 does not ask for an item name, and FoundItemReport has none, so matching cannot use it.	Member 1, Member 2
10	Status moved from the lost and found reports to FoundItemReport only; LostItemReport has no status.	LostItemReport, FoundItemReport, ReportStatus	FR-10 gives statuses to found items; FR-02 gives lost reports none, and a lost report is never CLAIMED or RETURNED.	Member 2
11	Removed RELEASED from ClaimStatus.	ClaimStatus, ReleaseRecord	FR-05, FR-12 and FR-13 only give Pending Verification, Approved and Rejected. The handover is already recorded by ReleaseRecord and by the item's RETURNED status.	Member 2
12	Removed update() and withdraw() from LostItemReport; edit() and withdraw(): boolean exist only on FoundItemReport.	LostItemReport, FoundItemReport	FR-02 only allows submitting a lost report; FR-08 (edit or withdraw) is about found reports.	Member 2, Member 3
13	Added UML notes: VerificationRecord.decision is only APPROVED or REJECTED; DisputeCase.resolve() approves the winning claim and rejects every other one; all claims in a dispute are for the same found item.	VerificationRecord, DisputeCase, Claim	Makes FR-15 and FR-16 visible in the diagram.	Member 2

2. Naming and responsibility differences resolved:

Item	Version A	Version B	Final decision
Edit operation	LostItemReport.update() (Member 1)	FoundItemReport.edit() (Member 2)	edit() on FoundItemReport only; update() removed (row 12)
Location attribute	lastSeenLocation (Member 1)	foundLocation (Member 2)	location in ItemReport (the place where the item was lost or found)
Date attribute	dateLost (Member 1)	dateTimeFound (Member 2)	eventDate in ItemReport (the date lost, or the date and time found)
Users who report items	ItemOwner and Finder (Member 1)	—	CampusMember with role names owner and finder
AUS ID in a claim	User.ausId (Member 1)	Claim.claimantAusId (Member 3)	Keep both: claimantAusId is the value the claimant types in and the officer compares (FR-05, FR-11, FR-12)
Private details	LostItemReport.privateDetails (Member 1)	Claim.identifyingDetails (Member 3)	Keep both: one is written in the lost report, the other is given when claiming
Email used for notifications	User.email (Member 1)	LostItemReport.contactEmail (Member 1) and Notification.recipientEmail (Member 2)	Keep contactEmail and recipientEmail (see section 6)

3. Attribute and method consistency changes:

Class	Change	Reason
ItemReport	Holds reportId, category, description, location, eventDate and submittedOn, and the operations submit() and attachPhoto().	Moved up from both report classes.
LostItemReport	Keeps privateDetails and contactEmail, and adds getPublicDetails(): String. itemName, status, update() and withdraw() removed.	getPublicDetails() hides the private details from other users (FR-02). Rows 9, 10 and 12.
FoundItemReport	Keeps color, status, edit(description, color, location), withdraw(): boolean, getStatus(), updateStatus() and the private isEditable().	FR-06, FR-08, FR-10.
CampusMember	submitFoundReport(report: FoundItemReport): boolean now takes a parameter.	Same style as reportLostItem(report: LostItemReport). Raised by Member 2 and Member 3.
CampusSecurityOfficer	approveClaim() and rejectClaim() removed; viewPendingClaims() and viewClaimHistory() added.	Rows 4 and 5.
User	createAccount() added; changePassword() removed.	FR-01.
ClaimStatus	RELEASED removed.	Row 11.

4. Inheritance review:

Relationship	Kept or removed	Reason
CampusMember → User	Kept	A campus member is a user that logs in with an AUS email (UC-01, FR-01).
CampusSecurityOfficer → User	Kept	An officer is a user that logs in the same way.
ItemOwner → User, Finder → User	Replaced	The two subclasses were merged into CampusMember (row 3).
LostItemReport → ItemReport, FoundItemReport → ItemReport	Added	Both are item reports: a lost or found report can be used wherever an item report is expected (substitutability).
ReportStatus, ClaimStatus	No inheritance	They are enumerations, not parent classes.

5. Associations and multiplicities review:

Association	Final name and multiplicities	Reason
CampusMember (owner) — LostItemReport	reports, 1 to 0..*	An owner can report many lost items; each report has one owner.
CampusMember (finder) — FoundItemReport	submits, 1 to 0..*	Member 2's association, now with the finder role name.
CampusMember (owner) — Claim	submits, 1 to 0..*	Member 3's association, now with the owner role name (FR-05).
Notification — CampusMember (owner)	is sent to, 0..* to 1	FR-04 emails the owner of the lost report.
ItemReport ◆— ItemPhoto	has, 1 to 0..3 (composition)	Moved from both report classes (FR-02, FR-07).
CampusMember ⇢ ItemSearch	dependency	Was an association (row 7).
FoundItemReport ⇢ ReportStatus	dependency	Only found reports have a status (row 10).
Other associations from Member 2 and Member 3	Unchanged	All are named, have both multiplicities and were confirmed in the reviews.

6. Conflicting design decisions:

Issue	Options	Decision	Reason
Photos owned by two classes	Separate LostItemPhoto and FoundItemPhoto classes, or one ItemPhoto owned by a parent class	Parent class ItemReport	All three reviews preferred it; it also removes the duplicated report attributes and gives a genuine inheritance relationship.
Owner and finder	Keep ItemOwner and Finder as subclasses with a note that one student can be both (Member 2), or merge them with role names (Member 3)	Merge into CampusMember	One object cannot be an instance of two subclasses, so a note cannot fix it. The merge carries out Member 2's idea that owner and finder are roles of one account.
Who receives the match notification	Owner only (FR-04), or also the finder (UC-09)	Owner only	FR-04 sends the email to the owner; the finder sees matches through the found report. Member 1 first suggested both and agreed with Member 2's explanation.
Contact email	Remove LostItemReport.contactEmail and Notification.recipientEmail and use User.email (Member 1, Member 3), or keep them (Member 2)	Keep them	D2 Lost Item Reports stores the owner's contact email, which may differ from the login email. Notification copies it into recipientEmail when it is created. Keeping it keeps the class model consistent with the DFD.
Status for lost reports	Reuse ReportStatus, add a separate enumeration, or no status	No status	No requirement gives lost reports a status (FR-02); FR-10 is about found items.
ReleaseRecord	Keep as a class, or turn into attributes of Claim	Keep as a class	It has its own data and operation, only approved claims have one (FR-14), and it is the moment the item becomes RETURNED. Member 2 and Member 3 agreed.
RELEASED claim status	Keep it, or let the claim stay APPROVED	Remove it	No requirement mentions it, and the handover is recorded elsewhere (row 11).
DISPUTED claim status	Add it, or leave claims PENDING_VERIFICATION until the dispute is resolved	Leave it out	No requirement mentions a disputed status (Member 3's decision, accepted).
Boundary class for the email service	Add an EmailGateway boundary class, or let Notification.send() represent it	No boundary class	Notification.send() is enough at this level; screens and external interfaces are left to design.

7. Final model for the consolidated diagram (for Member 2):

Packages:

Package	Classes
Users and Search	User, CampusMember, CampusSecurityOfficer, ItemSearch
Reports	ItemReport, LostItemReport, FoundItemReport, ItemPhoto, ReportStatus
Matching and Notification	MatchingService, MatchSuggestion, Notification
Claims	Claim, ClaimStatus, VerificationRecord, DisputeCase, ReleaseRecord

Classes that changed (draw the others exactly as in the individual diagrams):

Class	Attributes	Operations
<<entity>> *User* (abstract)	- ausId: String; - name: String; - email: String; - passwordHash: String	+ createAccount(email: String, password: String): boolean; + login(email: String, password: String): boolean; + logout(): void
<<entity>> CampusMember	- phone: String	+ reportLostItem(report: LostItemReport): boolean; + submitFoundReport(report: FoundItemReport): boolean; + startSearch(): ItemSearch
<<entity>> CampusSecurityOfficer	- badgeNumber: String; - office: String	+ viewPendingClaims(): List<Claim>; + viewClaimHistory(): List<VerificationRecord>

Class	Attributes	Operations
<<entity>> *ItemReport* (abstract)	- reportId: String; - category: String; - description: String; - location: String; - eventDate: Date; - submittedOn: Date	+ submit(): boolean; + attachPhoto(photo: ItemPhoto): boolean
<<entity>> LostItemReport	- privateDetails: String; - contactEmail: String	+ getPublicDetails(): String
<<entity>> FoundItemReport	- color: String; - status: ReportStatus	+ edit(description: String, color: String, location: String): boolean; + withdraw(): boolean; + getStatus(): ReportStatus; + updateStatus(newStatus: ReportStatus): void; - isEditable(): boolean
<<enumeration>> ClaimStatus	PENDING_VERIFICATION; APPROVED; REJECTED	—

Relationships:

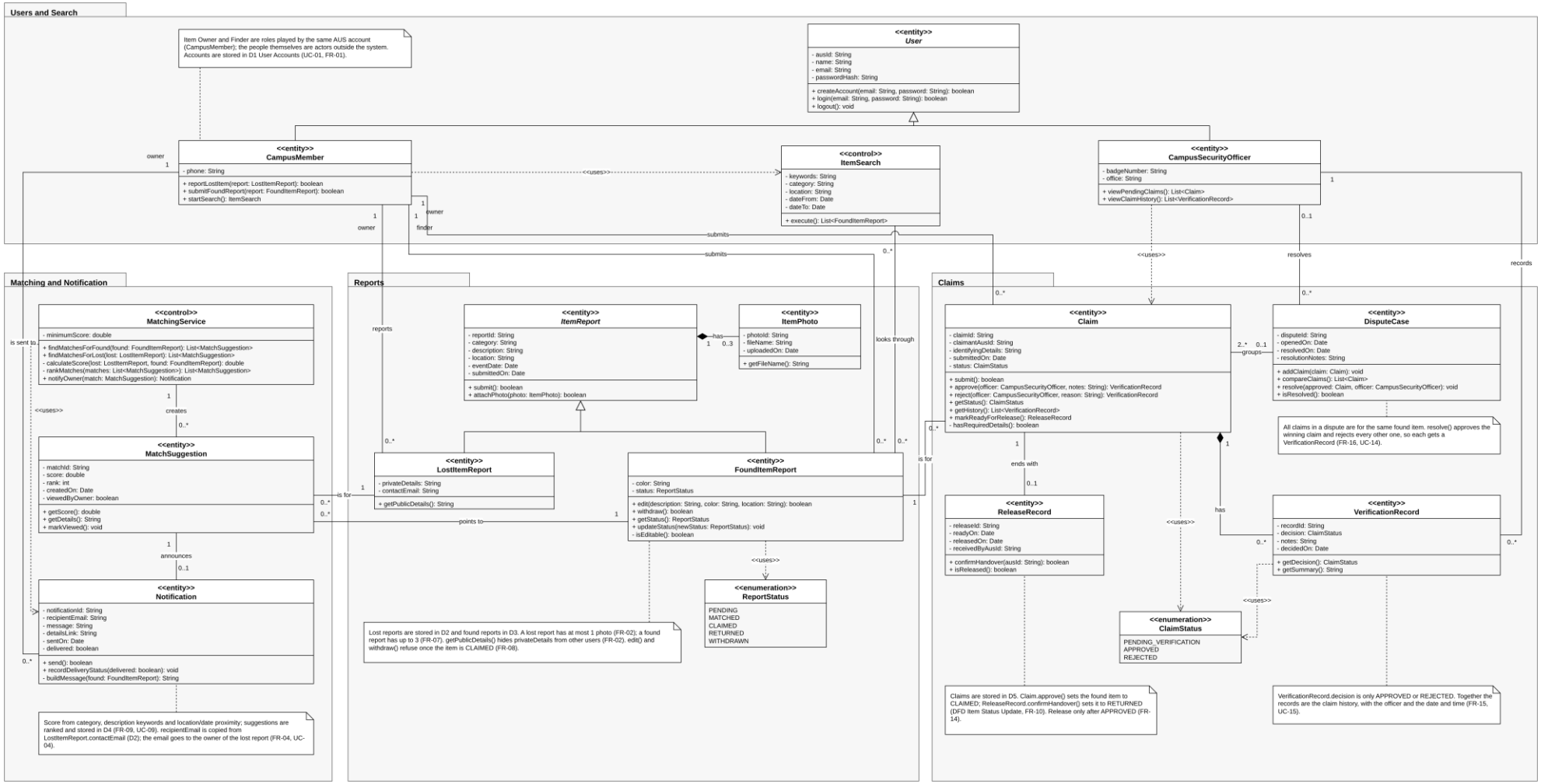
From	Relationship	To	Name / role	Multiplicity (From : To)
CampusMember	Generalization	User	—	—
CampusSecurityOfficer	Generalization	User	—	—
LostItemReport	Generalization	ItemReport	—	—
FoundItemReport	Generalization	ItemReport	—	—
CampusMember	Association	LostItemReport	reports (role: owner)	1 : 0..*
CampusMember	Association	FoundItemReport	submits (role: finder)	1 : 0..*
CampusMember	Association	Claim	submits (role: owner)	1 : 0..*
Notification	Association	CampusMember	is sent to (role: owner)	0..* : 1
ItemReport	Composition (filled diamond at ItemReport)	ItemPhoto	has	1 : 0..3
MatchSuggestion	Association	LostItemReport	is for	0..* : 1
MatchSuggestion	Association	FoundItemReport	points to	0..* : 1
MatchingService	Association	MatchSuggestion	creates	1 : 0..*
Notification	Association	MatchSuggestion	announces	0..1 : 1
ItemSearch	Association	FoundItemReport	looks through	0..* : 0..*
Claim	Association	FoundItemReport	is for	0..* : 1
Claim	Composition (filled diamond at Claim)	VerificationRecord	has	1 : 0..*
VerificationRecord	Association	CampusSecurityOfficer	records	0..* : 1
DisputeCase	Association	Claim	groups	0..1 : 2..*
CampusSecurityOfficer	Association	DisputeCase	resolves	0..1 : 0..*
Claim	Association	ReleaseRecord	ends with	1 : 0..1
CampusMember	Dependency (dashed arrow)	ItemSearch	uses	—
CampusSecurityOfficer	Dependency	Claim	uses (viewPendingClaims)	—
FoundItemReport	Dependency	ReportStatus	uses	—
Claim	Dependency	ClaimStatus	uses	—
VerificationRecord	Dependency	ClaimStatus	uses	—
MatchingService	Dependency	Notification	uses (notifyOwner)	—

UML notes to include:

1. ItemPhoto: a lost report has at most 1 photo (FR-02); a found report has up to 3 (FR-07).
2. CampusMember: the Item Owner and Finder actors are roles played by the same AUS account; the people themselves are actors outside the system.

3. Claim.approve() sets the found item to CLAIMED; ReleaseRecord.confirmHandover() sets it to RETURNED (DFD flow Item Status Update, FR-10). edit() and withdraw() refuse once the item is CLAIMED (FR-08).
4. VerificationRecord.decision is only APPROVED or REJECTED.
5. DisputeCase: all grouped claims are for the same found item; resolve() approves the winning claim and rejects every other one, so each gets a VerificationRecord (FR-16).
6. Notification.recipientEmail is copied from LostItemReport.contactEmail (D2).
7. The existing notes from the three individual diagrams (data stores, FR-14 release rule, claim history, matching score).

Consolidated UML Class Diagram:



Class Model Consistency and Traceability Check:

Member 3 checked the consolidated class model against the use cases, the functional requirements and the Level-0 DFD data stores, and checked every relationship in the consolidated diagram against the integration notes and the handout requirements.

1. Use cases vs classes (every use case must be supported by at least one class):

Use Case	Classes that support it	Covered?
UC-01 Log In	User, CampusMember, CampusSecurityOfficer	Yes
UC-02 Report Lost Item	CampusMember (owner), LostItemReport, ItemReport, ItemPhoto	Yes
UC-03 Search Found Items	CampusMember, ItemSearch, FoundItemReport	Yes
UC-04 View Match Notification	Notification, MatchSuggestion, CampusMember (owner)	Yes
UC-05 Submit Claim	CampusMember (owner), Claim, ClaimStatus, FoundItemReport	Yes
UC-06 Report Found Item	CampusMember (finder), FoundItemReport, ItemReport	Yes
UC-07 Attach Photo to Report	ItemReport (attachPhoto()), ItemPhoto	Yes
UC-08 Edit or Withdraw Found Item Report	FoundItemReport (edit(), withdraw(), isEditable()), ReportStatus	Yes
UC-09 Generate Match Suggestions	MatchingService, MatchSuggestion, LostItemReport, FoundItemReport, Notification	Yes
UC-10 Check Item Status	FoundItemReport (getStatus()), ReportStatus	Yes
UC-11 View Pending Claims	CampusSecurityOfficer (viewPendingClaims()), Claim, FoundItemReport	Yes
UC-12 Approve Claim	Claim (approve()), VerificationRecord, ReleaseRecord, CampusSecurityOfficer	Yes
UC-13 Reject Claim	Claim (reject()), VerificationRecord, CampusSecurityOfficer	Yes
UC-14 Resolve Disputed Claim	DisputeCase, Claim, VerificationRecord, CampusSecurityOfficer	Yes
UC-15 View Claim History	CampusSecurityOfficer (viewClaimHistory()), VerificationRecord	Yes

All 15 use cases have at least one class.

2. Functional requirements vs classes:

Functional Requirement	Classes (and attributes or methods) that satisfy it	Covered?
FR-01	User (email, passwordHash, createAccount(), login())	Yes
FR-02	LostItemReport (privateDetails, getPublicDetails()), ItemReport (category, description, location, eventDate, submit()), ItemPhoto (max 1 photo, see note)	Yes
FR-03	ItemSearch (keywords, category, location, dateFrom, dateTo, execute())	Yes
FR-04	Notification (recipientEmail, detailsLink, send()), MatchingService.notifyOwner()	Yes
FR-05	Claim (claimantAusId, identifyingDetails, submit()), ClaimStatus.PENDING_VERIFICATION	Yes
FR-06	FoundItemReport (color), ItemReport (category, description, location, eventDate)	Yes
FR-07	ItemReport ◆— ItemPhoto (0..3), attachPhoto()	Yes
FR-08	FoundItemReport.edit(), withdraw(), isEditable() (refuses once CLAIMED, see note)	Yes
FR-09	MatchingService (calculateScore(), rankMatches()), MatchSuggestion (score, rank)	Yes
FR-10	FoundItemReport.getStatus(), ReportStatus	Yes
FR-11	CampusSecurityOfficer.viewPendingClaims(), Claim (claimantAusId, identifyingDetails) linked to FoundItemReport	Yes
FR-12	Claim.approve(), ClaimStatus.APPROVED, VerificationRecord	Yes
FR-13	Claim.reject(), ClaimStatus.REJECTED, VerificationRecord	Yes
FR-14	Claim.markReadyForRelease() (only when APPROVED), ReleaseRecord	Yes
FR-15	VerificationRecord (decision, decidedOn) linked to CampusSecurityOfficer, viewClaimHistory()	Yes
FR-16	DisputeCase (compareClaims(), resolve()) groups 2..* claims	Yes

All 16 functional requirements are covered.

3. Data stores (DFD) vs classes:

Data Store	Class(es) that hold this data	Attributes cover the data described?
D1 User Accounts	User, CampusMember, CampusSecurityOfficer	Yes, User has ausId, email and passwordHash
D2 Lost Item Reports	LostItemReport, ItemReport, ItemPhoto	Yes, category, description, location, date, photo, private details and contact email
D3 Found Item Reports	FoundItemReport, ItemReport, ItemPhoto, ReportStatus	Yes, details, photos and status
D4 Match Suggestions	MatchSuggestion	Yes, the lost/found pair and the score
D5 Claims	Claim, ClaimStatus, VerificationRecord, DisputeCase, ReleaseRecord	Yes, claims, decisions, disputes and the history

Every data store has a class, and every entity class except Notification is saved in a data store (ItemSearch and MatchingService are control classes, so they aren't stored). Notification is not kept in any data store: P5 passes it to the external Email / Notification Service (DFD flows Match Notification Request and Delivery Status).

4. Relationships and multiplicities in the consolidated diagram:

I checked every line in Consolidated_Class_Diagram.drawio against section 7 of Integration_Notes.md. All 26 relationships are there with the right type, name and multiplicities.

Association	Name	Multiplicities	Justification from the requirements	Correct?
CampusMember → User	generalization	—	A campus member logs in with an AUS account (UC-01, FR-01)	Yes
CampusSecurityOfficer → User	generalization	—	An officer logs in the same way (UC-01, FR-01)	Yes
LostItemReport → ItemReport	generalization	—	A lost report is a kind of item report (FR-02)	Yes
FoundItemReport → ItemReport	generalization	—	A found report is a kind of item report (FR-06)	Yes
CampusMember (owner) — LostItemReport	reports	1 to 0..*	One owner can report many items, each report has one owner (FR-02)	Yes
CampusMember (finder) — FoundItemReport	submits	1 to 0..*	One finder can hand in many items (FR-06)	Yes
CampusMember (owner) — Claim	submits	1 to 0..*	One owner can make many claims, each claim is from one person (FR-05)	Yes
Notification — CampusMember (owner)	is sent to	0..* to 1	The email goes to the owner of the lost report (FR-04)	Yes
ItemReport ◆— ItemPhoto	has	1 to 0..3	Up to 3 photos, and a photo can't exist without its report (FR-02, FR-07)	Yes
MatchingService — MatchSuggestion	creates	1 to 0..*	Matching makes ranked suggestions (FR-09)	Yes
MatchSuggestion — LostItemReport	is for	0..* to 1	Each suggestion has one lost report (FR-09)	Yes
MatchSuggestion — FoundItemReport	points to	0..* to 1	Each suggestion has one found report (FR-09)	Yes
Notification — MatchSuggestion	announces	0..1 to 1	One email per match (FR-04)	Yes
ItemSearch — FoundItemReport	looks through	0..* to 0..*	A search can return many found items (FR-03)	Yes
Claim — FoundItemReport	is for	0..* to 1	Many people can claim the same item, each claim is for one item (FR-05, FR-16)	Yes
Claim ◆— VerificationRecord	has	1 to 0..*	Every decision is kept in the claim's history (FR-15)	Yes

Association	Name	Multiplicities	Justification from the requirements	Correct?
VerificationRecord — CampusSecurityOfficer	records	0..* to 1	The history shows which officer decided (FR-15)	Yes
DisputeCase — Claim	groups	0..1 to 2..*	A dispute needs at least 2 claims for the same item (UC-14, FR-16)	Yes
CampusSecurityOfficer — DisputeCase	resolves	0..1 to 0..*	The officer decides the dispute (FR-16)	Yes
Claim — ReleaseRecord	ends with	1 to 0..1	Only approved claims get released (FR-14)	Yes
CampusMember ⇢ ItemSearch	uses (dependency)	—	A search isn't saved anywhere, so it's not a lasting link (FR-03)	Yes
CampusSecurityOfficer ⇢ Claim	uses (dependency)	—	viewPendingClaims() returns claims (FR-11)	Yes
FoundItemReport ⇢ ReportStatus	uses (dependency)	—	Type of the status attribute (FR-10)	Yes
Claim ⇢ ClaimStatus	uses (dependency)	—	Type of the status attribute (FR-05)	Yes
VerificationRecord ⇢ ClaimStatus	uses (dependency)	—	Type of the decision attribute (FR-15)	Yes
MatchingService ⇢ Notification	uses (dependency)	—	notifyOwner() creates the notification (FR-04)	Yes

Small things I noticed (not wrong, just cosmetic): the MatchingService ⇢ Notification and VerificationRecord ⇢ ClaimStatus arrows don't have the «uses» label like the other dependencies, and the 0..* next to ItemSearch on "looks through" is a bit hidden by a line crossing. These don't change the model. Member 2 fixed both before submission (the two «uses» labels were added and the 0..* label was moved), and the diagram in this report is the fixed version.

5. Handout requirements for the consolidated diagram:

Requirement	Result
Approved classes from the individual models included	Yes, all 17 classes in Class_Inventory.md are in the diagram, each one only once
Attributes and methods shown	Yes, every class has its attributes and methods (enumerations show their values)
Visibility shown	Yes, + and - on every attribute and method
Data types, parameters and return types shown	Yes, for example + approve(officer: CampusSecurityOfficer, notes: String): VerificationRecord
Meaningful, named associations with multiplicities	Yes, all 16 associations have a name and multiplicities at both ends, and role names (owner, finder) where CampusMember plays a role
Genuine inheritance only	Yes, 4 generalizations, all real is-a: CampusMember and CampusSecurityOfficer are users, LostItemReport and FoundItemReport are item reports. User and ItemReport are abstract (italic)
Helpful UML notes	Yes, 6 notes (roles of CampusMember, photos and data stores, matching score and email, status updates and release rule, disputes, claim history)
Readable and professionally organized	Yes, the classes are grouped into 4 packages (Users and Search, Reports, Matching and Notification, Claims). It's big, so in the report it goes on its own landscape page
Created with a UML/modeling tool	Yes, draw.io (.drawio and .png in Class_Diagram/)
Consistent with Labs 1-4	Yes, after the changes in section 6 below

6. Changes needed to earlier labs:

File	Change Needed	Reason
Project/Data_Flow_Modeling/Level_0_DFD/Level_0_Flows.md	P6 Process and Decide Claims: "FR-05, FR-11 to FR-15" changed to "FR-05, FR-11 to FR-16"	FR-16 was added in Lab 4 but P6 still said FR-11 to FR-15
Project/Data_Flow_Modeling/DFD_Consistency_Check.md	Same change for P6 in section 5	Same thing
Project/Requirements/Use_Cases.md	UC-09 description: the owner of the lost report gets the email, and the finder sees the match through their found report	The team decided the notification goes to the owner only (Integration_Notes.md section 6, FR-04). UC-09 said it notifies the finder
Project/Requirements/Use_Cases.md	UC-10 description: added Withdrawn to the list of statuses	ReportStatus has WITHDRAWN (FR-08 lets the finder withdraw a report), and D3 already lists it
Project/Requirements/Functional_Requirements.md	FR-10: added Withdrawn to the list of statuses	Same reason
Project/Requirements/Functional_Requirements.md	No change: UC-14 not having an FR was already fixed in Lab 4 (FR-16), and DisputeCase covers it	Just noting that I checked it
Project/Requirements/Use_Cases.md	No change to the actors: Item Owner and Finder stay as actors	In the class model they are roles of one CampusMember account, but in the use case model they are still two different actors (note on CampusMember)

Changes Made to Earlier Labs:

File	Change Made	Caused By
Project/Data_Flow_Modeling/Level_0_DFD/Level_0_Flows.md	P6 Process and Decide Claims now lists FR-05, FR-11 to FR-16.	FR-16 was added in Lab 4 but the P6 row still said FR-11 to FR-15. Found in the Lab 5 consistency check.
Project/Data_Flow_Modeling/DFD_Consistency_Check.md	Same change for P6 in section 5.	Same reason.
Project/Requirements/Use_Cases.md	UC-09 Generate Match Suggestions: the description now says the owner of the lost report gets the email and the finder sees the match through their found report (member table and consolidated table).	Lab 5 integration decision: the notification goes to the owner only (FR-04, Notification is sent to CampusMember as owner).
Project/Requirements/Use_Cases.md	UC-10 Check Item Status: added Withdrawn to the list of statuses (member table and consolidated table).	ReportStatus in the class model has WITHDRAWN (FR-08), and D3 Found Item Reports already lists it.
Project/Requirements/Functional_Requirements.md	FR-10: added Withdrawn to the list of statuses (member table and consolidated table).	Same reason.

No change was needed for UC-14, because FR-16 (added in Lab 4) already covers resolving disputed claims. The actors Item Owner and Finder stay in the use case model; in the class model they are roles played by one CampusMember account.